

Perancangan dan Analisis Algoritma

Minggu 8: BWB dan KMSL4B



Institut Teknologi Sepuluh Nopember

Fakultas Teknologi Elektro dan Informatika Cerdas

Departemen Teknik Informatika

Mata Kuliah: Perancangan dan Analisis Algoritma

Dosen Pengampu: Rully Soelaiman, S.Kom., M.Kom.

Nama : Ida Bagus Gde Dimas Sutha Maha

NRP : 5025241218

Kelas : B

Email : 5025241218@student.its.ac.id

Tanggal : 21 April 2026

Semester : Genap 2025/2026

Daftar Isi

1	BWB — Black and White Beads	3
1.1	Problem Description	3
1.1.1	Problem Statement	3
1.1.2	Input	3
1.1.3	Output	3
1.1.4	Batas Implementasi pada Program	3
1.2	Observasi	3
1.2.1	Ide Dasar	3
1.2.2	Relasi Rekursif	4
1.2.3	Penjabaran Sliding Window	5
1.2.4	Contoh Perhitungan	5
1.3	Source Code	6
1.4	Time Complexity	7
1.5	SPOJ Accepted Status	7
1.6	Pseudocode	7
2	KMSL4B — Roots of Polynomial	9
2.1	Problem Description	9
2.1.1	Problem Statement	9
2.1.2	Input	9
2.1.3	Output	9
2.1.4	Constraints	9
2.1.5	Sample Input	9
2.1.6	Sample Output	10
2.2	Observasi	10
2.2.1	Ide Dasar	10
2.2.2	Penjabaran Syarat Awal	10
2.2.3	Reduksi Koefisien	11
2.2.4	Normalisasi Koefisien	13
2.2.5	Contoh Reduksi Satu Langkah	13
2.3	Source Code	14

2.4	Time Complexity	15
2.5	SPOJ Accepted Status	16
2.6	Pseudocode	16

1. BWB — Black and White Beads

1.1. Problem Description

1.1.1 Problem Statement

Pada problem *BWB - Black and White Beads*, tujuan problem adalah menghitung banyak susunan manik-manik hitam dan putih yang valid. Berdasarkan statement problem, terdapat dua syarat utama, yaitu tidak boleh terdapat K manik hitam berturut-turut dan manik pertama tidak boleh berwarna hitam, sehingga bead pertama harus berwarna putih [1].

1.1.2 Input

Input diawali banyaknya test case. Untuk setiap test case diberikan dua bilangan bulat, yaitu n dan k .

1.1.3 Output

Untuk setiap test case, keluarkan banyak susunan valid sesuai aturan problem. Pada implementasi program yang dibuat, hasil dihitung modulo:

$$10^9 + 7$$

1.1.4 Batas Implementasi pada Program

Pada program yang digunakan:

$$1 \leq T \leq 10^6, \quad 1 \leq k \leq 100, \quad 1 \leq n \leq 10000$$

1.2. Observasi

1.2.1 Ide Dasar

Misalkan F_i menyatakan banyak susunan valid untuk i posisi yang tersisa setelah bead pertama ditetapkan berwarna putih. Pada soal, bead pertama tidak boleh berwarna hitam, sehingga bead pertama selalu pasti putih. Oleh karena itu, jika panjang total string adalah n , maka yang masih perlu dihitung hanya $n - 1$ posisi berikutnya.

Karena setiap posisi setelah bead pertama hanya dapat diisi dua warna, maka tanpa larangan apa pun banyak susunan panjang i adalah:

$$2^i$$

Untuk $i < k$, belum mungkin muncul k manik hitam berturut-turut. Maka semua susunan sepanjang i masih valid. Dengan demikian:

$$F_0 = 1, \quad F_1 = 2, \quad F_2 = 4, \quad \dots, \quad F_{k-1} = 2^{k-1}$$

Secara umum:

$$F_i = 2^i, \quad 0 \leq i < k$$

Sebagai contoh, untuk $k = 3$:

$$F_0 = 1, \quad F_1 = 2, \quad F_2 = 4$$

1.2.2 Relasi Rekursif

Untuk panjang $i \geq k$, tidak semua susunan valid, sehingga perlu dicari pola pembentukannya.

Sebuah susunan valid sepanjang i pasti berakhir dengan salah satu bentuk berikut:

$$W, \quad BW, \quad BBW, \quad \dots, \quad B^{k-1}W$$

Penjelasannya sebagai berikut:

1. Jika susunan valid berakhir dengan W , maka $i - 1$ posisi sebelumnya bebas berupa susunan valid panjang $i - 1$.
2. Jika susunan valid berakhir dengan BW , maka $i - 2$ posisi sebelumnya harus berupa susunan valid panjang $i - 2$.
3. Jika susunan valid berakhir dengan BBW , maka $i - 3$ posisi sebelumnya harus berupa susunan valid panjang $i - 3$.
4. Proses ini berlanjut sampai suffix $B^{k-1}W$, karena jika ada k buah B berturut-turut maka susunan menjadi tidak valid.

Dengan demikian, banyak susunan valid panjang i dapat dijabarkan sebagai:

$$F_i = \underbrace{F_{i-1}}_{\text{akhiran } W} + \underbrace{F_{i-2}}_{\text{akhiran } BW} + \underbrace{F_{i-3}}_{\text{akhiran } BBW} + \dots + \underbrace{F_{i-k}}_{\text{akhiran } B^{k-1}W}$$

Jadi relasi rekursifnya diperoleh dari penjabaran bentuk suffix:

$$F_i = F_{i-1} + F_{i-2} + \dots + F_{i-k}$$

Bentuk sigma-nya adalah:

$$F_i = \sum_{j=1}^k F_{i-j}$$

1.2.3 Penjabaran Sliding Window

Jika setiap kali nilai F_i dihitung dengan menjumlahkan ulang k suku sebelumnya, maka kompleksitas per langkah menjadi lebih besar. Oleh karena itu, pada program digunakan teknik *sliding window*.

Misalkan:

$$W_i = F_{i-1} + F_{i-2} + \cdots + F_{i-k}$$

maka dapat ditulis:

$$F_i = W_i$$

Selanjutnya diperhatikan window berikutnya:

$$W_{i+1} = F_i + F_{i-1} + F_{i-2} + \cdots + F_{i-k+1}$$

Jika dibandingkan dengan:

$$W_i = F_{i-1} + F_{i-2} + \cdots + F_{i-k}$$

maka terlihat bahwa:

- suku baru yang masuk ke dalam window adalah F_i ,
- suku lama yang keluar dari window adalah F_{i-k} .

Jadi, bentuk pembaruan window diperoleh langsung dari pergeseran jendela penjumlahan:

$$\boxed{W_{i+1} = W_i + F_i - F_{i-k}}$$

1.2.4 Contoh Perhitungan

Untuk $k = 3$:

$$F_0 = 1, \quad F_1 = 2, \quad F_2 = 4$$

Karena suffix valid yang mungkin adalah:

$$W, \quad BW, \quad BBW$$

maka:

$$F_3 = F_2 + F_1 + F_0 = 4 + 2 + 1 = 7$$

$$F_4 = F_3 + F_2 + F_1 = 7 + 4 + 2 = 13$$

$$F_5 = F_4 + F_3 + F_2 = 13 + 7 + 4 = 24$$

$$\left\{ \begin{array}{l} F_0 = 1 \\ F_1 = 2F_0 = 2 \\ F_2 = 2F_1 = 4 \\ F_3 = F_2 + F_1 + F_0 = 7 \\ F_4 = F_3 + F_2 + F_1 = 13 \\ F_5 = F_4 + F_3 + F_2 = 24 \end{array} \right.$$

1.3. Source Code

```
#include <stdio>
#define ll long long
const int MOD = 1000000007;
const int MAXN = 10000;
const int MAXK = 100;
int dp[MAXK + 1][MAXN];
void precom(){
    int lim = MAXN - 1;
    for (int k = 1; k <= MAXK; k++){
        dp[k][0] = 1;
        ll wind = 1; int i = 1;
        while (i < k && i <= lim){
            dp[k][i] = (2LL * dp[k][i - 1]) % MOD;
            wind = (wind + dp[k][i]) % MOD;
            i++;
        }
        while (i <= lim){
            dp[k][i] = (int)wind;
            wind = (wind + dp[k][i] - dp[k][i - k]) % MOD;
            if (wind < 0) wind += MOD;
            i++; }}}

int main(){
    precom();
    int tc; scanf("%d", &tc);
    while (tc--){
        int n, k;
        scanf("%d %d", &n, &k);
        printf("%d\n", dp[k][n - 1]);
    }
    return 0;
}
```

1.4. Time Complexity

Precomputation dilakukan untuk semua:

$$1 \leq k \leq MAXK$$

dan:

$$0 \leq i < MAXN$$

Setiap pasangan (k, i) dihitung satu kali, sehingga:

$$\text{precomputation} = O(MAXK \cdot MAXN)$$

Setelah precomputation selesai, setiap test case hanya mengambil jawaban dari tabel:

$O(1)$

Jadi total kompleksitas waktunya:

$$O(MAXK \cdot MAXN + t)$$

dengan t adalah banyak test case.

Untuk memori, program menggunakan tabel:

$$dp[MA XK + 1][MA XN]$$

Sehingga:

$$O(MAXK \cdot MAXN)$$

1.5. SPOJ Accepted Status

35694660		2026-04-21 06:40:21	Black and White beads	accepted edit · delete · flag	0.01	5.3M	CPP
----------	---	------------------------	-----------------------	----------------------------------	------	------	-----

1.6. Pseudocode

Algorithm 1 BWB - Black and White Beads

```
1: procedure PRECOM
2:    $lim \leftarrow MAXN - 1$ 
3:   for  $k \leftarrow 1$  to  $MAXK$  do
4:      $dp[k][0] \leftarrow 1$ 
5:      $wind \leftarrow 1$ 
6:      $i \leftarrow 1$ 
7:     while  $i < k$  and  $i \leq lim$  do
8:        $dp[k][i] \leftarrow 2 \cdot dp[k][i - 1]$ 
9:        $wind \leftarrow wind + dp[k][i]$ 
10:       $i \leftarrow i + 1$ 
11:    end while
12:    while  $i \leq lim$  do
13:       $dp[k][i] \leftarrow wind$ 
14:       $wind \leftarrow wind + dp[k][i] - dp[k][i - k]$ 
15:      if  $wind < 0$  then
16:         $wind \leftarrow wind + MOD$ 
17:      end if
18:       $i \leftarrow i + 1$ 
19:    end while
20:  end for
21: end procedure
22: procedure MAIN
23:   PRECOM
24:   input( $tc$ )
25:   while  $tc > 0$  do
26:     input( $n, k$ )
27:     output( $dp[k][n - 1]$ )
28:      $tc \leftarrow tc - 1$ 
29:   end while
30: end procedure
```

2. KMSL4B — Roots of Polynomial

2.1. Problem Description

2.1.1 Problem Statement

Pada problem *KMSL4B - Roots of Polynomial*, diberikan polinomial berderajat paling tinggi 20 dan diminta menentukan apakah semua akarnya berada pada *open unit disc*:

$$|z| < 1$$

di bidang kompleks [2].

2.1.2 Input

Input diawali banyaknya test case. Untuk setiap test case:

1. diberikan derajat polinomial n ,
2. kemudian diberikan koefisien:

$$p_0, p_1, \dots, p_n$$

2.1.3 Output

Untuk setiap test case:

- keluarkan 1 jika semua akar polinomial berada pada $|z| < 1$,
- keluarkan 0 jika ada minimal satu akar dengan $|z| \geq 1$

2.1.4 Constraints

Derajat maksimum polinomial adalah:

$$n \leq 20$$

2.1.5 Sample Input

```
2
2
1 2 1
2
0.5 1 1
```

2.1.6 Sample Output

0
1

2.2. Observasi

2.2.1 Ide Dasar

Diberikan polinomial:

$$p(x) = p_0 + p_1x + p_2x^2 + \cdots + p_nx^n$$

Pada program, koefisien disimpan langsung sebagai:

$$a[0] = p_0, \quad a[1] = p_1, \quad \dots, \quad a[n] = p_n$$

Karena output yang diminta hanya berupa keputusan biner, penulis tidak perlu menghitung semua akar secara eksplisit. Pendekatan yang dipakai adalah reduksi koefisien bertahap berdasarkan kriteria kestabilan terhadap lingkaran satuan, sejalan dengan ide pada *Jury stability criterion* [4] dan *Schur-Cohn test* [6].

2.2.2 Penjabaran Syarat Awal

Misalkan:

$$p(x) = a_0 + a_1x + \cdots + a_nx^n$$

Jika akar-akarnya adalah:

$$z_1, z_2, \dots, z_n$$

maka polinomial dapat dijabarkan menjadi:

$$p(x) = a_n(x - z_1)(x - z_2) \cdots (x - z_n)$$

Sekarang substitusi $x = 0$ dilakukan:

$$p(0) = a_0 = a_n(0 - z_1)(0 - z_2) \cdots (0 - z_n)$$

Sehingga:

$$a_0 = a_n(-z_1)(-z_2) \cdots (-z_n)$$

Ambil nilai mutlak kedua ruas:

$$|a_0| = |a_n| \cdot |(-z_1)(-z_2) \cdots (-z_n)|$$

Karena nilai mutlak mengabaikan tanda, maka:

$$|a_0| = |a_n| \cdot |z_1 z_2 \cdots z_n|$$

Bagi kedua ruas dengan $|a_n|$:

$$\left| \frac{a_0}{a_n} \right| = |z_1 z_2 \cdots z_n|$$

Jika semua akar berada di dalam open unit disc, maka:

$$|z_1| < 1, \quad |z_2| < 1, \quad \dots, \quad |z_n| < 1$$

Karena hasil kali beberapa bilangan yang masing-masing bernilai kurang dari 1 juga bernilai kurang dari 1, maka:

$$|z_1 z_2 \cdots z_n| < 1$$

Akibatnya:

$$\left| \frac{a_0}{a_n} \right| < 1$$

Kalikan kedua ruas dengan $|a_n|$:

$$|a_0| < |a_n|$$

Jadi syarat awal yang diperoleh dari penjabaran akar adalah:

$$\boxed{|a_0| < |a_n|}$$

2.2.3 Reduksi Koefisien

Syarat di atas hanya syarat perlu, sehingga polinomial masih harus direduksi.

Bentuk polinomial baliknya adalah:

$$p^*(x) = x^n p\left(\frac{1}{x}\right)$$

Sekarang $p\left(\frac{1}{x}\right)$ dijabarkan:

$$p\left(\frac{1}{x}\right) = a_0 + a_1 \left(\frac{1}{x}\right) + a_2 \left(\frac{1}{x}\right)^2 + \cdots + a_n \left(\frac{1}{x}\right)^n$$

Kalikan seluruh ruas dengan x^n :

$$p^*(x) = a_0 x^n + a_1 x^{n-1} + a_2 x^{n-2} + \cdots + a_n$$

Jika urutan ditulis dari pangkat kecil ke besar:

$$p^*(x) = a_n + a_{n-1}x + \cdots + a_1x^{n-1} + a_0x^n$$

Kemudian dibentuk:

$$a_np(x) - a_0p^*(x)$$

Jabarkan suku pertama:

$$a_np(x) = a_na_0 + a_na_1x + a_na_2x^2 + \cdots + a_na_nx^n$$

Jabarkan suku kedua:

$$a_0p^*(x) = a_0a_n + a_0a_{n-1}x + a_0a_{n-2}x^2 + \cdots + a_0a_0x^n$$

Kurangkan kedua bentuk tersebut:

$$a_np(x) - a_0p^*(x)$$

$$= (a_na_0 - a_0a_n) + (a_na_1 - a_0a_{n-1})x + (a_na_2 - a_0a_{n-2})x^2 + \cdots + (a_na_n - a_0a_0)x^n$$

Suku konstanta adalah:

$$a_na_0 - a_0a_n = 0$$

Karena suku konstanta nol, seluruh bentuk dapat dibagi dengan x . Setelah dibagi dengan x , koefisien yang tersisa menjadi:

$$(a_na_1 - a_0a_{n-1}) + (a_na_2 - a_0a_{n-2})x + \cdots + (a_na_n - a_0a_0)x^{n-1}$$

Jika koefisien baru ini dinamai $b_0, b_1, \dots, b_{n-1}$, maka:

$$b_0 = a_na_1 - a_0a_{n-1}$$

$$b_1 = a_na_2 - a_0a_{n-2}$$

$$b_2 = a_na_3 - a_0a_{n-3}$$

$$\vdots$$

$$b_{n-1} = a_na_n - a_0a_0$$

Dari pola tersebut diperoleh:

$$\boxed{b_i = a_na_{i+1} - a_0a_{n-i-1}} \quad 0 \leq i < n$$

Setelah itu:

1. array b disalin kembali ke a ,
2. derajat dikurangi satu,
3. proses yang sama diulang pada polinomial baru.

Dengan demikian proses reduksi berjalan:

$$n \rightarrow n - 1 \rightarrow n - 2 \rightarrow \dots \rightarrow 0$$

2.2.4 Normalisasi Koefisien

Sebelum reduksi, penulis melakukan normalisasi.

Misalkan:

$$mx = \max(|a_0|, |a_1|, \dots, |a_n|)$$

Kemudian semua koefisien dibagi dengan mx :

$$a_0 \leftarrow \frac{a_0}{mx}, \quad a_1 \leftarrow \frac{a_1}{mx}, \quad \dots, \quad a_n \leftarrow \frac{a_n}{mx}$$

Jika polinomial semula adalah:

$$p(x) = a_0 + a_1x + \dots + a_nx^n$$

maka setelah dibagi mx menjadi:

$$\frac{1}{mx}p(x) = \frac{a_0}{mx} + \frac{a_1}{mx}x + \dots + \frac{a_n}{mx}x^n$$

Karena yang berubah hanya faktor pengali konstanta, akar polinomial tidak berubah. Jadi normalisasi hanya menjaga skala angka tetap stabil.

2.2.5 Contoh Reduksi Satu Langkah

Untuk polinomial derajat 3:

$$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3$$

Mula-mula diperiksa:

$$|a_0| < |a_3|$$

Jika syarat ini terpenuhi, maka:

$$a_3p(x) = a_3a_0 + a_3a_1x + a_3a_2x^2 + a_3a_3x^3$$

$$a_0p^*(x) = a_0a_3 + a_0a_2x + a_0a_1x^2 + a_0a_0x^3$$

Kurangkan:

$$a_3p(x) - a_0p^*(x) \\ = (a_3a_0 - a_0a_3) + (a_3a_1 - a_0a_2)x + (a_3a_2 - a_0a_1)x^2 + (a_3a_3 - a_0a_0)x^3$$

Karena suku konstanta nol, hasilnya dibagi dengan x , sehingga koefisien baru menjadi:

$$b_0 = a_3a_1 - a_0a_2$$

$$b_1 = a_3a_2 - a_0a_1$$

$$b_2 = a_3a_3 - a_0a_0$$

Indeks	Koefisien Baru
b_0	$a_3a_1 - a_0a_2$
b_1	$a_3a_2 - a_0a_1$
b_2	$a_3a_3 - a_0a_0$

Tabel 1: Pembentukan koefisien baru pada satu langkah reduksi.

2.3. Source Code

```
#include <stdio.h>
#include <math.h>
#define MAXN 22
typedef long double ld;
ld a[MAXN], b[MAXN];
ld eps = 1e-12;

void bagi(int n){
    ld mx = 0;
    for(int i = 0; i <= n; i++){
        if(fabs1(a[i]) > mx){mx = fabs1(a[i]);}

        if(mx == 0) {return;}
        for(int i = 0; i <= n; i++){a[i] = a[i] / mx;}}

int cek(int n){
    while(n > 0){
        bagi(n);
        if(fabs1(a[0]) + eps >= fabs1(a[n])){return 0;}
        for(int i = 0; i < n; i++){b[i] = a[n] * a[i + 1] - a[0] * a[n - i - 1];}
        for(int i = 0; i < n; i++){a[i] = b[i];} n--;}
    if(fabs1(a[0]) < eps){return 0;}
    return 1;
```

```

}

int main(){
    int tc, n;
    scanf("%d", &tc);
    while(tc--){
        scanf("%d", &n);
        for(int i = 0; i <= n; i++){scanf("%Lf", &a[i]);}
        printf("%d\n", cek(n));}
    return 0;
}

```

2.4. Time Complexity

Pada setiap iterasi dengan derajat saat ini m , program melakukan:

1. normalisasi koefisien: $O(m)$
2. pembentukan array baru: $O(m)$
3. penyalinan kembali: $O(m)$

Jadi satu iterasi membutuhkan:

$$O(m)$$

Karena derajat turun satu demi satu:

$$n, n - 1, n - 2, \dots, 1$$

maka total waktu:

$$O(n) + O(n - 1) + O(n - 2) + \dots + O(1)$$

Jumlah tersebut merupakan deret:

$$1 + 2 + 3 + \dots + n$$

Dengan rumus jumlah bilangan asli:

$$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2}$$

maka kompleksitas waktunya adalah:

$$\boxed{O(n^2)}$$

Memori yang digunakan hanya dua array:

$$a[MAN], \quad b[MAN]$$

Sehingga kompleksitas memorinya:

$O(n)$

Karena pada problem ini:

$$n \leq 20$$

maka secara praktis waktu per test case tetap kecil.

2.5. SPOJ Accepted Status

Gde Dimas	Roots of polynomial	accepted edit delete it	0	5.1M	C++ 4.3.2
-----------	---------------------	---	---	------	--------------

2.6. Pseudocode

Algorithm 2 KMSL4B - Roots of Polynomial

```
1: procedure BAGI( $n$ )
2:    $mx \leftarrow 0$ 
3:   for  $i \leftarrow 0$  to  $n$  do
4:     if  $|a[i]| > mx$  then
5:        $mx \leftarrow |a[i]|$ 
6:     end if
7:   end for
8:   if  $mx = 0$  then
9:     return
10:  end if
11:  for  $i \leftarrow 0$  to  $n$  do
12:     $a[i] \leftarrow a[i]/mx$ 
13:  end for
14: end procedure
15: procedure CEK( $n$ )
16:  while  $n > 0$  do
17:    BAGI( $n$ )
18:    if  $|a[0]| + eps \geq |a[n]|$  then
19:      return 0
20:    end if
21:    for  $i \leftarrow 0$  to  $n - 1$  do
22:       $b[i] \leftarrow a[n] \cdot a[i + 1] - a[0] \cdot a[n - i - 1]$ 
23:    end for
24:    for  $i \leftarrow 0$  to  $n - 1$  do
25:       $a[i] \leftarrow b[i]$ 
26:    end for
27:     $n \leftarrow n - 1$ 
28:  end while
29:  if  $|a[0]| < eps$  then
30:    return 0
31:  end if
32:  return 1
33: end procedure
34: procedure MAIN
35:  input( $tc$ )
36:  while  $tc > 0$  do
37:    input( $n$ )
38:    input( $a[0], a[1], \dots, a[n]$ )
39:    output(CEK( $n$ ))
40:     $tc \leftarrow tc - 1$ 
41:  end while
42: end procedure
```

Pustaka

- [1] SPOJ, *BWB - Black and White Beads*. <https://www.spoj.com/problems/BWB/>
- [2] SPOJ, *KMSL4B - Roots of polynomial*. <https://www.spoj.com/problems/KMSL4B/>
- [3] Kenneth H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed., McGraw-Hill, 2019.
- [4] Wikipedia, *Jury stability criterion*. https://en.wikipedia.org/wiki/Jury_stability_criterion
- [5] GeeksforGeeks, *Sliding Window Technique*. <https://www.geeksforgeeks.org/dsa/window-sliding-technique/>
- [6] Wikipedia, *Schur-Cohn test*. https://en.wikipedia.org/wiki/Schur-Cohn_test